Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

КУРСОВОЙ ПРОЕКТ

МЕТОДЫ ИМИТАЦИОННОГО МОДЕЛИРОВАНИЯ

по дисциплине «Архитектура программных систем»

Выполнил Верниковский А. В.
студент гр. 5130904/40101

Руководитель Гончаров А. В.

Санкт-Петербург
2026

Оглавление

Введение **2**

Исходные данные **2**

Маппинг **3**

Архитектура приложения **3**

Диаграмма классов 4

Сиквенс диаграмма 6

BPMN диаграмма 6

Блок-схема 7

Вывод законов распределения **8**

Равномерный закон ИЗ2 8

Экспоненциальный закон ПЗ1 9

Ограничения и требуемые характеристики **9**

Модульная структура **10**

Описание работы программы **11**

Пошаговый режим 11

Автоматический режим 12

Результаты работы и анализ **13**

Вывод **15**

Введение

Цель курсового проекта — модель вычислительной системы, которая сохраняет структуру объекта и его поведение на выбранном уровне детализации. По модели нужно судить о том, как система ведёт себя во времени: как быстро заявка проходит контур, когда ей отказывают и насколько заняты приборы.

У реальной вычислительной системы много состояний и связей. Разбирать их все сразу неудобно, поэтому в модели оставляют те свойства, которые нужны для поставленного вопроса. Аналитическая схема даёт формулы для простых случаев. Имитационная схема нужна там, где интервалы случайны, правила очереди не сводятся к одной формуле, а приборы разбиты на группы. В этой работе выбран имитационный метод. Система представлена как система массового обслуживания: источники порождают заявки, буфер их хранит, приборы обслуживают, два диспетчера задают порядок постановки и выбора.

Предмет модели — вычислительный контур теплицы, вариант 22. Датчики температуры, влажности, освещённости и вредителей независимо формируют заявки на расчёт. Заявка ждёт в общей очереди и считается на воркере своего контура. Нагреватель, лампа и распылитель приборами не являются: команда на них уходит уже после успешного расчёта. Программа на Python воспроизводит календарь событий и собирает сводные таблицы, по которым сравниваются число воркеров, ёмкость буфера и интенсивность потока.

Исходные данные

Вариант 22. ИБ ИЗ2 ПЗ1 Д1ОЗ2 Д1ОО4 Д2П2 Д2Б4 ОР1 ОД1.

Источники. ИБ — бесконечные. Каждый датчик после съёма сразу планирует следующий, поэтому поток не иссякает. Источников четыре, по контурам: И1 — температура, И2 — влажность, И3 — свет, И4 — вредители. Меньший номер контура важнее.

Закон генерации. ИЗ2 — равномерный закон $U[a, b]$ для интервала между съёмами. Так задаётся джиттер опроса датчика.

Приборы. ПЗ1 — экспоненциальное время обслуживания $\mathrm{Exp}(\mu)$. Прибор — worker-процесс, а не исполнительный механизм теплицы. На каждый контур приходится группа воркеров. По умолчанию это пары Р1–Р2, Р3–Р4, Р5–Р6 и Р7–Р8.

Дисциплина буферизации. Д1ОЗ2 — постановка в хвост общей очереди. При изъятии заявки остальные сдвигаются, пустых мест в середине не остаётся.

Дисциплина отказа. Д1ОО4 — выбивание последней поступившей заявки, то есть той, которая простояла в буфере меньше всех. Если буфер полон, эта заявка удаляется, на её место встаёт новая, и удаление фиксируется как отказ.

Дисциплина выбора заявки. Д2Б4 — приоритет источника, по одной заявке. Сначала берётся контур с меньшим номером, а среди заявок одного контура — самая старая.

Дисциплина выбора прибора. Д2П2 — кольцо внутри группы показателя. Свободные воркеры нужного знака дельты подбираются так, чтобы сумма дельт закрыла отклонение климата от оптимума. При равенстве модулей порядок задаёт указатель кольца. Заявка влажности не назначается на воркеры температуры.

Отображение. ОР1 — автоматический режим, сводные таблицы и подбор числа заявок. ОД1 — пошаговый режим: календарь событий, содержимое буфера и текущее состояние приборов.

Маппинг

Контур принимает съёмы датчиков и превращает их в команды актуаторам. Если воркеры группы заняты, заявка ждёт в общей памяти расчёта.

1. Источник. Источник — датчик контура. Он порождает бесконечный поток заявок. Интервал между съёмами равномерный: разброс сети и АЦП лежит в окне $[a, b]$.

2. Заявка. Заявка — задание на один съём: проверить показание, посчитать воздействие, записать факт. В ней хранятся номер источника, номер съёма и время поступления. Это не капля воды и не градус температуры, а единица работы воркера.

3. Буфер. Буфер — общая ограниченная очередь на edge-узле. Новая заявка встаёт в хвост. Если места нет, выбивается заявка, пришедшая последней, и отказ попадает в статистику. Уже накопленный расчёт, особенно по температуре, сохраняет место: свежий съём ещё почти не ждал.

4. Диспетчер постановки. Диспетчер постановки срабатывает в момент генерации. Он работает только с буфером и приборов не знает. Свободное место заполняется по Д1ОЗ2, переполнение разрешается по Д1ОО4.

5. Диспетчер выбора. Диспетчер выбора срабатывает, когда в буфере есть заявка и свободен воркер её группы. Он берёт одну заявку по Д2Б4 и назначает свободные приборы группы по кольцу Д2П2. Выдача из буфера — отдельное событие календаря с небольшой задержкой, чтобы очередь была видна между постановкой и назначением.

6. Прибор. Прибор — воркер своего контура. В каждый момент он считает не больше одной заявки, время счёта экспоненциальное. После завершения климат сдвигается на дельту этого воркера: вентилятор или батарея, вытяжка или полив, зонты или UV-лампы, химикаты или ловушки.

Программа ведёт число сгенерированных, отклонённых и обслуженных заявок, времена ожидания и обслуживания и загрузку каждого воркера. В пошаговом режиме видны календарь, буфер, занятость приборов и указатель кольца.

Архитектура приложения

Модель собрана из сервисов, которые обмениваются событиями, а не прямыми вызовами приборов из диспетчера постановки. Календарь задаёт модельное время: сервисы не ждут по часам стенного времени, иначе дисциплины очереди разъедутся. Ниже — диаграмма классов, последовательность обработки заявки, схема процесса по дорожкам и блок-схема того же потока.

Диаграмма классов

Класс `Simulation` держит календарь, буфер, оба диспетчера, источники, приборы и состояние климата. Диспетчер постановки связан только с буфером. Диспетчер выбора видит буфер и группы приборов. Заявка создаётся источником и обслуживается прибором.

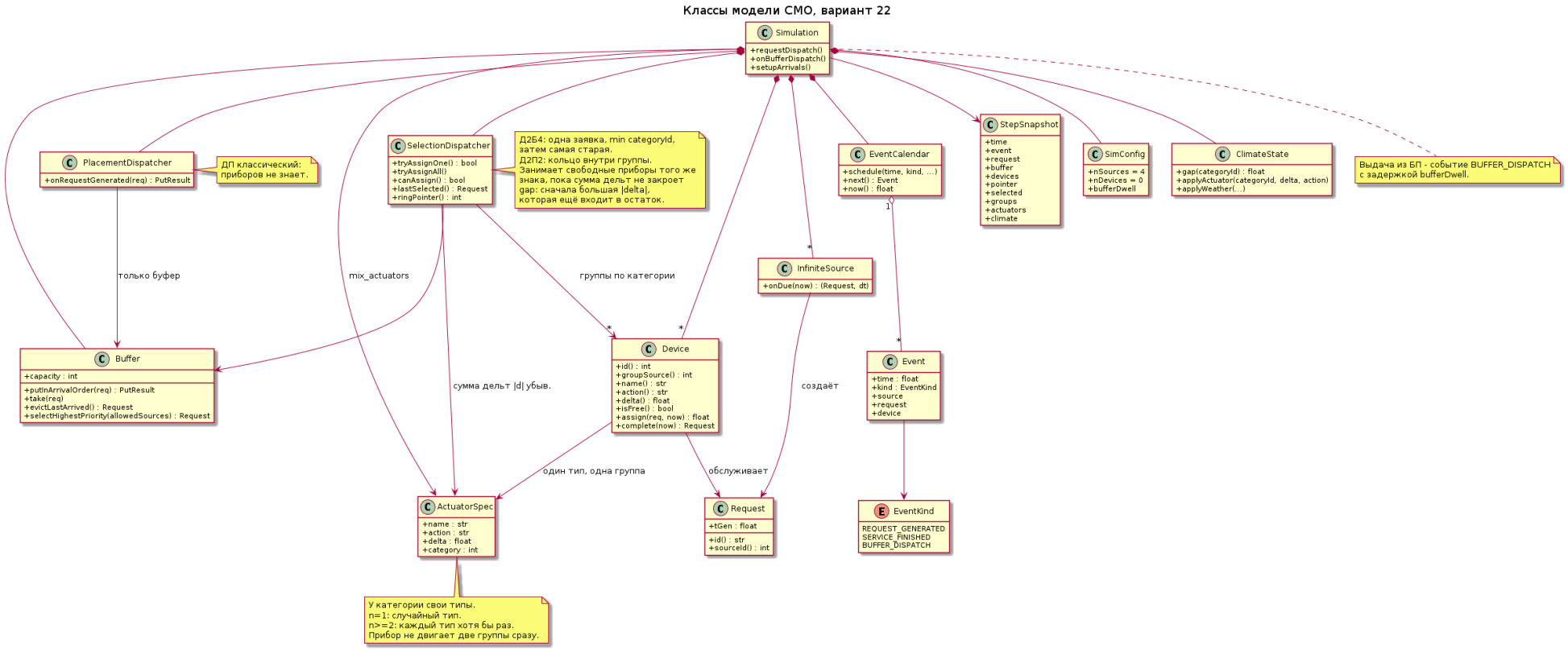


Рис. 1: Диаграмма классов модели, вариант 22

Сиквенс диаграмма

На диаграмме три фрагмента. В первом заявка контура И1 сразу уходит на свободный прибор группы. Во втором буфер полон, приборы группы заняты, и по Д1ООО4 выбивается последняя поступившая заявка. В третьем после освобождения прибора из буфера выдаётся одна заявка: приоритет контура, затем самая старая.

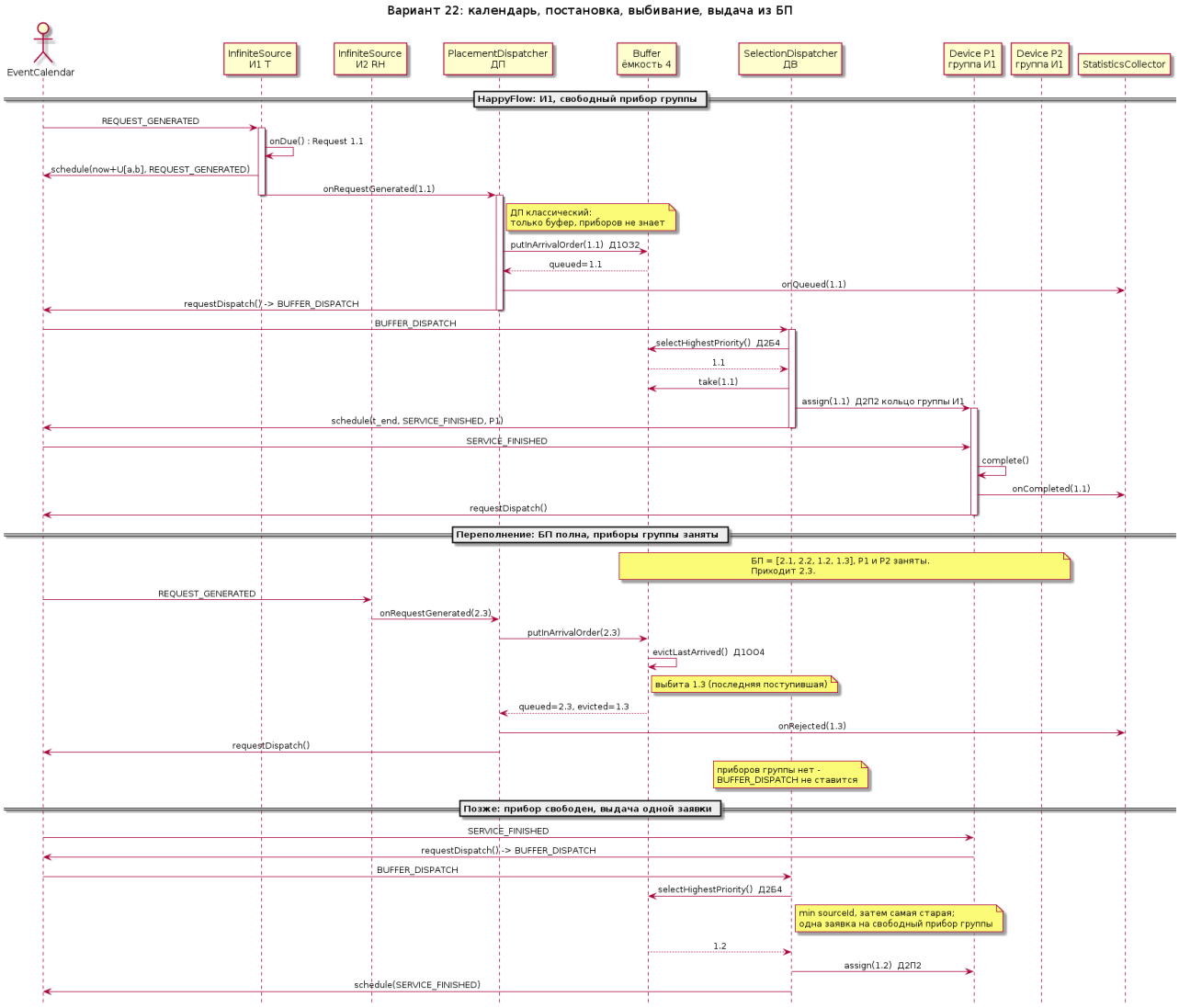


Рис. 2: Последовательность: постановка, отказ и выдача из буфера

BPMN диаграмма

Дорожки повторяют роли контура: датчик, диспетчер постановки, календарь, диспетчер выбора, воркер. Ветвь «буфер заполнен» — отказ по Д1ООО4. Ветвь «нет свободного воркера группы» оставляет заявку ждать следующего события.

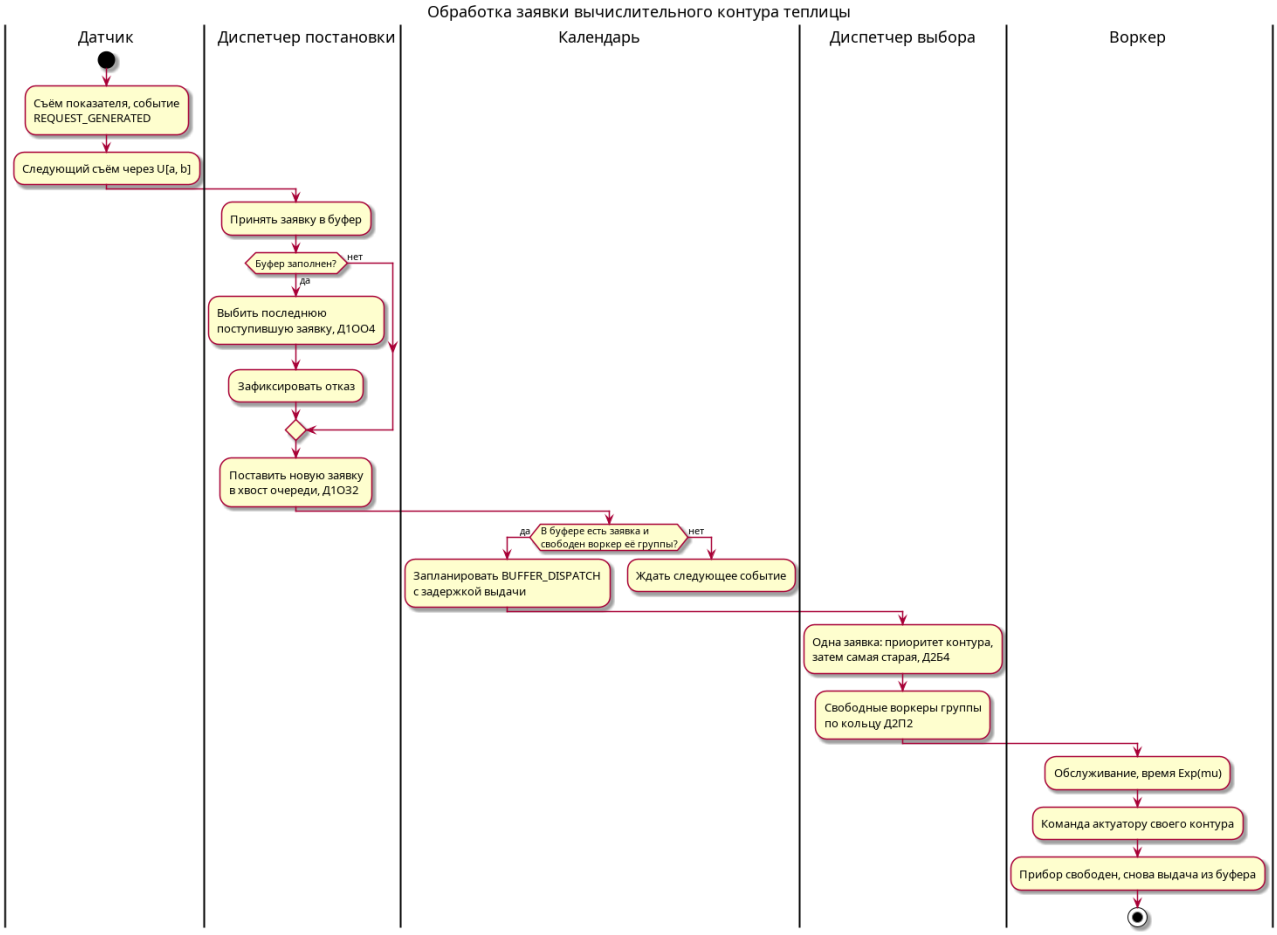


Рис. 3: Процесс обработки заявки по дорожкам

Блок-схема

Блок-схема задаёт тот же порядок в виде потока: генерация, постановка или выбивание, планирование выдачи, выбор одной заявки, обслуживание, команда актуатору и возврат к выдаче из буфера.

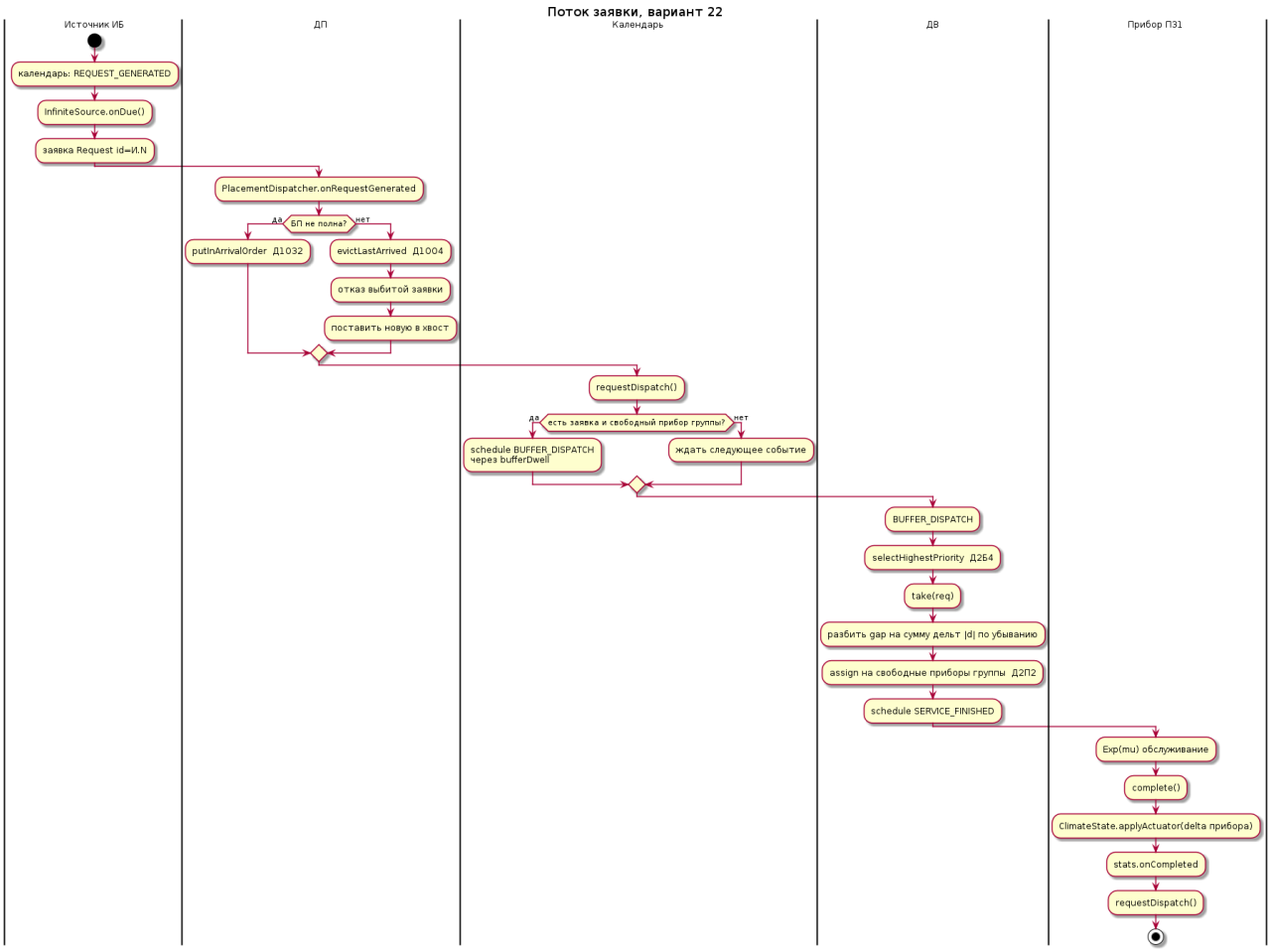


Рис. 4: Блок-схема прохождения заявки

Вывод законов распределения

В варианте два закона. Интервал генерации — равномерный, время обслуживания — экспоненциальное. Оба получаются из равномерного датчика $u \in (0, 1)$ обратным преобразованием функции распределения.

Равномерный закон ИЗ2

Пусть интервал между съёмами одного датчика лежит в $[a, b]$, $b > a$, и внутри окна нет предпочтительных точек. Плотность постоянна, а интеграл по окну равен единице:

$$f(x) = \begin{cases} \dfrac{1}{b-a}, & x \in [a, b], \\ 0, & \text{иначе.} \end{cases}$$

Функция распределения на отрезке линейна:

$$F(x) = \frac{x-a}{b-a}, \qquad a \le x \le b.$$

Математическое ожидание и дисперсия:

$$\mathbb{E}[X] = \frac{a+b}{2}, \qquad \mathbb{D}[X] = \frac{(b-a)^2}{12}.$$

Обратное преобразование: если u равномерно на $(0,1)$, то $x = a+(b-a)u$ имеет распределение $U[a,b]$. Именно так источник планирует следующий съём. При параметрах программы по умолчанию $a = 1$, $b = 3$, средний интервал одного датчика равен 2. Четыре датчика дают суммарную интенсивность $\lambda = 4/2 = 2$.

Экспоненциальный закон ПЗ1

Время обслуживания воркера не зависит от того, сколько оно уже длится. Этому соответствует плотность

$$f(t) = \mu e^{-\mu t}, \qquad t \geq 0,\ \mu > 0,$$

и функция распределения

$$F(t) = 1 - e^{-\mu t}.$$

Среднее и дисперсия:

$$\mathbb{E}[T] = \frac{1}{\mu}, \qquad \mathbb{D}[T] = \frac{1}{\mu^2}.$$

Из $F(t) = u$ получается формула генератора:

$$t = -\frac{\ln(1-u)}{\mu}.$$

Поскольку $1-u$ распределено так же, как u, в программе берётся $t = -\ln(u)/\mu$. По умолчанию $\mu = 0{,}55$, среднее обслуживание около 1,82 единицы модельного времени.

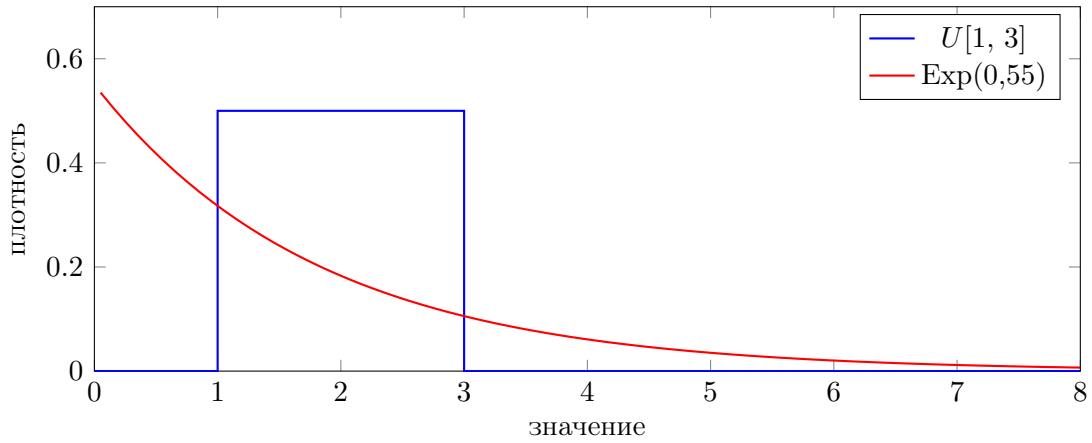


Рис. 5: Плотности ИЗ2 и ПЗ1 при параметрах программы по умолчанию

Ограничения и требуемые характеристики

Для выбора конфигурации приняты такие ограничения.

- Вероятность отказа $p_{\text{отк}}$ не выше 0,10.
- Загрузка приборов в среднем выше 0,90.

Время пребывания заявки в системе сверху не ограничено: расчёт может длиться долго, если так получается по показанию и по экспоненциальному обслуживанию. Отказ возможен только в буфере. Воркер либо свободен, либо занят, отдельной поломки прибора нет.

Число реализаций, нужное для относительной точности δ при доверительной вероятности α, оценивается по доле отказов:

$$N^* = \frac{t_\alpha^2 \, (1 - p)}{p \, \delta^2}, \tag{1}$$

где p — вероятность отказа, $t_\alpha = 1{,}643$ при $\alpha = 0{,}9$, $\delta = 0{,}1$.

Авторежим считает эту величину после прогона. На параметрах по умолчанию при $N = 100$ получено $p = 0{,}15$ и $N^* = 1530$. Подстановка в формулу (1) даёт то же число: $1{,}643^2 \cdot 0{,}85/(0{,}15 \cdot 0{,}01) \approx 1530$. Для p около 0,10 формула даёт примерно 2400 заявок. В сравнении конфигураций ниже каждый прогон содержит $N = 2000$ заявок: это тот же порядок, что и оценка по формуле.

Модульная структура

Программа написана на Python и запускается из корня проекта командой `./smo.py`. Режимы: `decode` — расшифровка кодов варианта, `step` — пошаговый календарь, `auto` — таблицы и подбор N.

Основные классы:

- `InfiniteSource` — датчик. По событию календаря создаёт заявку и планирует следующий интервал $U[a, b]$.
- `Request` — заявка: источник, номер съёма, времена генерации, начала и конца обслуживания.
- `Buffer` — общая очередь. Методы постановки в порядке прихода, изъятия и выбивания последней поступившей.
- `Device` — воркер группы. Назначение заявки, экспоненциальное время, завершение и накопленная занятость.
- `PlacementDispatcher` — диспетчер постановки. Проверяет буфер и не обращается к приборам.
- `SelectionDispatcher` — диспетчер выбора. Одна заявка по приоритету контура и кольцо приборов внутри группы.
- `EventCalendar` — очередь особых событий в модельном времени.
- `StatisticsCollector` — счётчики источников и приборов для таблиц ОР1.
- `Simulation` — сборка контура: связывает календарь, буфер, диспетчеры, климат и статистику.

Состояние климата и актуаторы лежат рядом с приборами, но в очередь СМО не входят. Они меняются уже после `complete`.

Описание работы программы

Ниже — результаты конечных прогонов: календарь пошагового режима и сводные таблицы авторежима.

Пошаговый режим

Команда:

```
./smo.py step -n 12 --seed 42
```

Параметры — значения по умолчанию: четыре датчика, восемь воркеров, буфер на 4 места, $a = 1$, $b = 3$, $\mu = 0{,}55$. В таблице календаря оставлены все 38 шагов этого прогона. События названы короче, чем в консоли, из свободных приборов перечислены только занятые. Числа совпадают с выводом программы. Доля отказов в этом коротком прогоне равна нулю: буфер не переполнялся. Ненулевой отказ виден в авторежиме ниже.

Таблица 1: Календарь событий, `step -n 12`, зерно 42

Шаг	t	Событие	Заявка	БП	Приборы	Ук.	$p_{\text{отк}}$	N
1	1,733	генерация	3.1	3.1	—	1	0,000	1
2	2,026	генерация	1.1	3.1, 1.1	—	1	0,000	2
3	2,213	выдача из БП	1.1	3.1	P2=1.1	3	0,000	2
4	2,226	генерация	2.1	3.1, 2.1	P2=1.1	3	0,000	3
5	2,693	выдача из БП	2.1	3.1	P2=1.1, P4=2.1	5	0,000	3
6	2,719	конец обслуж.	1.1	3.1	P4=2.1	5	0,000	3
7	2,835	конец обслуж.	2.1	3.1	—	5	0,000	3
8	2,922	генерация	4.1	3.1, 4.1	—	5	0,000	4
9	3,173	выдача из БП	3.1	4.1	P6=3.1	7	0,000	4
10	3,653	выдача из БП	4.1	—	P6=3.1, P7=4.1, P8=4.1	8	0,000	4
11	3,667	конец обслуж.	3.1	—	P7=4.1, P8=4.1	8	0,000	4
12	3,908	генерация	1.2	1.2	P7=4.1, P8=4.1	8	0,000	5
13	4,388	выдача из БП	1.2	—	P1=1.2, P7=4.1, P8=4.1	2	0,000	5
14	4,659	генерация	3.2	3.2	P1=1.2, P7=4.1, P8=4.1	2	0,000	6
15	4,785	конец обслуж.	4.1	3.2	P1=1.2, P8=4.1	2	0,000	6
16	4,821	конец обслуж.	1.2	3.2	P8=4.1	2	0,000	6
17	4,901	генерация	4.2	3.2, 4.2	P8=4.1	2	0,000	7
18	5,102	генерация	2.2	3.2, 4.2, 2.2	P8=4.1	2	0,000	8
19	5,139	выдача из БП	2.2	3.2, 4.2	P4=2.2, P8=4.1	5	0,000	8
20	5,619	выдача из БП	3.2	4.2	P4=2.2, P6=3.2, P8=4.1	7	0,000	8
21	6,099	выдача из БП	4.2	—	P4=2.2, P6=3.2, P7=4.2, P8=4.1	8	0,000	8
22	6,371	генерация	2.3	2.3	P4=2.2, P6=3.2, P7=4.2, P8=4.1	8	0,000	9
23	6,454	генерация	1.3	2.3, 1.3	P4=2.2, P6=3.2, P7=4.2, P8=4.1	8	0,000	10
24	6,851	выдача из БП	1.3	2.3	P1=1.3, P4=2.2, P6=3.2, P7=4.2, P8=4.1	2	0,000	10
25	6,920	конец обслуж.	1.3	2.3	P4=2.2, P6=3.2, P7=4.2, P8=4.1	2	0,000	10
26	6,921	генерация	3.3	2.3, 3.3	P4=2.2, P6=3.2, P7=4.2, P8=4.1	2	0,000	11
27	7,037	конец обслуж.	4.1	2.3, 3.3	P4=2.2, P6=3.2, P7=4.2	2	0,000	11
28	7,331	выдача из БП	2.3	3.3	P3=2.3, P4=2.2, P6=3.2, P7=4.2	4	0,000	11
29	7,380	конец обслуж.	4.2	3.3	P3=2.3, P4=2.2, P6=3.2	4	0,000	11
30	7,607	конец обслуж.	2.2	3.3	P3=2.3, P6=3.2	4	0,000	11
31	7,778	генерация	4.3	3.3, 4.3	P3=2.3, P6=3.2	4	0,000	12
32	7,811	выдача из БП	3.3	4.3	P3=2.3, P5=3.3, P6=3.2	6	0,000	12
33	8,291	выдача из БП	4.3	—	P3=2.3, P5=3.3, P6=3.2, P7=4.3, P8=4.3	8	0,000	12
34	9,022	конец обслуж.	3.2	—	P3=2.3, P5=3.3, P7=4.3, P8=4.3	8	0,000	12
35	9,251	конец обслуж.	4.3	—	P3=2.3, P5=3.3, P8=4.3	8	0,000	12
36	9,670	конец обслуж.	3.3	—	P3=2.3, P8=4.3	8	0,000	12
37	10,119	конец обслуж.	4.3	—	P3=2.3	8	0,000	12
38	11,090	конец обслуж.	2.3	—	—	8	0,000	12

По этому же прогону программа печатает сводку. $T_{\text{преб}}$, $T_{\text{БП}}$ и $T_{\text{обсл}}$ — средние пребывание, ожидание и обслуживание. $D_{\text{БП}}$ и $D_{\text{обсл}}$ — дисперсии ожидания и обслуживания.

$K_{\text{исп}}$ — доля времени, когда прибор был занят, на длине реализации $T = 11{,}0897$.

Таблица 2: Характеристики источников, `step -n 12`

И	N	$p_{\text{отк}}$	$T_{\text{преб}}$	$T_{\text{БП}}$	$T_{\text{обсл}}$	$D_{\text{БП}}$	$D_{\text{обсл}}$
1	3	0,0000	0,6904	0,3547	0,3357	0,0152	0,0366
2	3	0,0000	2,6107	0,4881	2,1226	0,1421	2,2388
3	3	0,0000	3,0149	1,0966	1,9183	0,0598	1,4124
4	3	0,0000	2,9781	0,8142	2,1639	0,0817	0,7938

Таблица 3: Приборы, `step -n 12`. Всего заявок 12, отказов 0, $\rho \approx 0{,}455$

P1	P2	P3	P4	P5	P6	P7	P8
0,0452	0,0456	0,3389	0,2353	0,1676	0,3513	0,3040	0,4699

Кольцо видно по столбцу указателя: после выдачи он сдвигается на следующий прибор группы. Заявка 4.1 занимает сразу P7 и P8: отклонение контура вредителей режется на сумму дельт двух свободных воркеров. Заявка 3.1 ждёт в буфере, пока не освободится прибор света, и уходит раньше более поздних заявок старших контуров только после них. Это Д2Б4: сначала меньший номер источника.

Автоматический режим

Команда:

```
./smo.py auto --seed 42
```

Программа делает прогон, оценивает p и N^* по формуле (1) и увеличивает N, пока оценка не стабилизируется. Если N^* превышает внутренний потолок 800, перебор останавливается. Так вышло в этом запуске: уже первая итерация дала $N^* = 1530$.

Таблица 4: Подбор N в авторежиме, параметры по умолчанию

Итерация	N	$p_{\text{отк}}$	N^*
1	100	0,1500	1530

Таблица 5: Источники после авторежима, $N = 100$

И	N	$p_{\text{отк}}$	$T_{\text{преб}}$	$T_{\text{БП}}$	$T_{\text{обсл}}$
1	24	0,2917	2,2341	0,2421	1,9920
2	25	0,0000	2,9020	0,7721	2,1299
3	27	0,1852	1,9185	0,5403	1,3782
4	24	0,1250	7,4855	4,9508	2,5346

Таблица 6: Приборы после авторежима. Заявок 100, отказов 15, $p = 0{,}1500$, $T = 53{,}9645$

P1	P2	P3	P4	P5	P6	P7	P8
0,2876	0,3399	0,6026	0,3841	0,1951	0,3668	0,6433	0,4167

Отказы распределены неравномерно. Контур И1 важнее остальных, но при переполнении Д1ОО4 выбивает последнюю пришедшую заявку, и ею часто оказывается свежий съём температуры: у И1 доля отказов 0,29. Контур И4 ждёт дольше всех, $T_{\text{БП}} = 4{,}95$. Загрузка приборов далека от 0,90: при $\rho \approx 0{,}46$ пул слишком свободен, хотя буфер на 4 места уже даёт 15 отказов из 100.

Результаты работы и анализ

Условные стоимости компонентов:

- медленный воркер, среднее обслуживание 7 (то есть $\mu = 1/7$), — 30 000 у. е.;
- быстрый воркер, среднее обслуживание 3,5 ($\mu = 1/3{,}5$), — 55 000 у. е.;
- одно место буфера — 10 000 у. е.;
- обработанная заявка — 500 у. е.;
- отклонённая заявка — 500 у. е.;
- единица времени пребывания заявки в системе — 2 у. е.

Воркеров не больше 13, мест в буфере не больше 13. Штраф 500 у. е. одинаков для обслуженной и отклонённой заявки. При фиксированном N он сдвигает все строки на одну и ту же величину и не меняет выбор конфигурации. В столбце «Итог» оставлены затраты, которые от конфигурации зависят:

$$C = c\,n + 10\,000\,m + 2\,T_{\text{преб}}. \tag{2}$$

Здесь n — число воркеров, c — цена одного воркера выбранного типа, m — число мест буфера, $T_{\text{преб}}$ — среднее время пребывания обслуженной заявки.

Датчиков четыре, по одному на контур. Воркеры делятся между контурами поровну, поэтому сравниваются 4, 8 и 12 приборов. Суммарная интенсивность обозначена λ: средний интервал одного датчика равен $4/\lambda$, окно — $U[2/\lambda,\ 6/\lambda]$, с тем же отношением границ, что и окно по умолчанию $U[1, 3]$. Загрузка в таблицах — среднее $K_{\text{исп}}$ по всем воркерам. Каждый прогон: $N = 2000$, зерно 42, задержка выдачи из буфера 0,48.

При $\lambda = 1$ быстрые воркеры либо не укладываются в отказ 10%, либо простаивают.

Таблица 7: Быстрые воркеры, $\lambda = 1$

Датчики	Воркеры	Буфер	λ	Загрузка	$p_{\text{отк}}$	$T_{\text{преб}}$	Итог
4	4	4	1	0,65	0,2520	7,09	260014,18
4	4	10	1	0,73	0,1435	12,50	320025,00
4	8	4	1	0,43	0,0030	4,20	480008,40

Четыре быстрых воркера даже с десятью местами буфера дают $p_{\text{отк}} = 0{,}14$. Восемь воркеров снимают отказы, но загрузка падает до 0,43, а итог вырастает до 480 008 у. е.

Восемь медленных воркеров дают ту же суммарную интенсивность обслуживания, что и четыре быстрых ($8/7 = 4/3{,}5$), и стоят 240 000 против 220 000. Отказ при этом ниже: на каждый контур приходится два прибора, и занятость одного не закрывает группу. Граница 10% проходит между шестью и семью местами буфера.

Таблица 8: Медленные воркеры, $\lambda = 1$. Серым выделена допустимая конфигурация с наименьшим итогом

Датчики	Воркеры	Буфер	λ	Загрузка	$p_{\text{отк}}$	$T_{\text{преб}}$	Итог
4	4	13	1	0,83	0,4880	29,94	250059,88
4	8	6	1	0,74	0,1120	10,85	300021,70
4	8	7	1	0,76	0,0995	11,49	310022,97
4	8	13	1	0,79	0,0560	15,15	370030,30
4	12	4	1	0,62	0,0265	8,68	400017,36

Четыре медленных воркера при полном буфере из 13 мест всё ещё отклоняют почти половину заявок. Шесть мест при восьми воркерах дают $p = 0{,}112$, это выше порога. Семь мест дают $p = 0{,}0995$ при итоге 310 022,97. Дополнительные места и набор из двенадцати воркеров уменьшают отказ, но загрузка не доходит до 0,90, а итог растёт. Поток $\lambda = 1$ слишком слабый, чтобы одновременно удержать отказ не выше 10% и загрузку выше 90%.

Интенсивность увеличена до $\lambda = 3$. Медленные воркеры в этот порог отказа не входят: даже 12 приборов и 13 мест оставляют $p > 0{,}55$. Загрузка поднимается к 0,80–0,85, но ценой массового выбивания заявок.

Таблица 9: Медленные воркеры, $\lambda = 3$. Ограничение по отказу не выполняется

Датчики	Воркеры	Буфер	λ	Загрузка	$p_{\text{отк}}$	$T_{\text{преб}}$	Итог
4	4	13	3	0,85	0,8185	29,43	250058,85
4	8	13	3	0,80	0,6775	19,54	370039,08
4	12	8	3	0,77	0,5740	12,94	440025,87
4	12	13	3	0,80	0,5560	16,44	490032,89

Быстрые воркеры при $\lambda = 3$ отклоняют меньше, чем медленные при том же числе приборов, но 10% всё равно не достигается. Самая низкая доля отказа в этой серии — 0,41 при двенадцати воркерах, и такой ряд стоит от 700 000 у. е. при загрузке около 0,56.

Таблица 10: Быстрые воркеры, $\lambda = 3$. Ограничение по отказу не выполняется

Датчики	Воркеры	Буфер	λ	Загрузка	$p_{\text{отк}}$	$T_{\text{преб}}$	Итог
4	4	4	3	0,67	0,7300	7,25	260014,49
4	4	13	3	0,75	0,6875	16,36	350032,73
4	8	8	3	0,67	0,4810	8,00	520016,01
4	12	13	3	0,56	0,4125	10,59	790021,18

Четыре группы приборов не делят одну общую очередь обслуживания: заявка света не занимает воркер температуры. Пока одна группа переполнена, другая успевает простаивать, поэтому средняя загрузка отстаёт от загрузки самого занятого контура. Поднять среднее выше 0,90, не выходя за $p = 0{,}10$, в пределах 13 воркеров и 13 мест не удалось ни на $\lambda = 1$, ни на $\lambda = 3$.

Из строк с $p_{\text{отк}}$ не выше 0,10 наименьший итог у восьми медленных воркеров и семи мест буфера при $\lambda = 1$. Это 310 022,97 у. е., загрузка 0,76, время пребывания 11,49. Соседняя строка с шестью местами дешевле на 10 000, но отказ 0,112 уже выше порога. Тринадцатое место поднимает загрузку лишь до 0,79 и добавляет 60 000.

Вывод

По варианту 22 построена имитационная модель вычислительного контура теплицы и программа на Python, которая ведёт календарь событий и сводные таблицы. Датчики четырёх контуров дают равномерный поток заявок, общая очередь ставит их в хвост и при переполнении выбивает последнюю пришедшую, диспетчер выбора отдаёт одну заявку по приоритету контура на воркеры группы.

Сравнение конфигураций проведено на прогонах по 2000 заявок. Ограничение по отказу 10% и ограничение по загрузке 90% вместе, при не более чем 13 воркерах и 13 местах буфера, не достигаются: плотный поток $\lambda = 3$ поднимает отказ выше 0,40, а добавление воркеров при $\lambda = 1$ оставляет их недогруженными. Среди конфигураций с отказом не выше 10% наименьшие затраты даёт набор из восьми медленных воркеров, по два на контур, и семи мест буфера при $\lambda = 1$. Затраты по формуле (2) составляют 310 022,97 у. е. при вероятности отказа 0,0995 и средней загрузке 0,76.